

Classes and Objects

Defining Classes

In Java everything is encapsulated under classes.

A class defines new data type. Once defined this new type can be used to create object of that type. Object is an instance of class.

A class is declared using **class** keyword.

A class contain both data and code that operate on that data.

The data or variables defined within a **class** are called **instance variables** and the code that operates on this data is known as **methods**.

Syntax:

```
class ClassName [extends SuperClassName]
{
    [fields declarations]
    [methods declarations]
}
```

Field Declaration

```
class ClassName
{
    Datatype variableName;
}
```

Method Declaration

```
class ClassName
{
    ReturnType methodName(Parameter List)
    {
        \\Body
    }
}
```

Example:

```
class Rectangle
{
    int length;
    int width;
    void getData(int x,int y)
    {
```

```
        length=x;
        width=y;
    }
}
```

Creating Objects

Object is an instance of a class. Class is a template or blueprint from which objects are created. So object is the instance(result) of a class.

Syntax:

1. Declare the Object

```
ClassName obectName;
```

2. Instantiate the Object

```
objectName=new ClassCostructorName();
```

1 and 2 Combined

```
ClassName objectName=new ClassConstructorName();
```

Example:

1. Rectangle r1;

```
r1=new Rectangle();
```

2. Rectangle r1=new Rectangle();

Full Example of Class and its Object

```
class Rectangle
{
    int length;
    int width;
    void getData(int x,int y)
    {
        length=x;
        width=y;
    }
}

class eg
{
    public static void main(String args[])
    {
        Rectangle r1=new Rectangle();
    }
}
```

```
        r1.getData(5,10);
    }
}
```

Static Members

The **static keyword** in java is used for memory management mainly.

The static can be:

1. variable (also known as class variable)
2. method (also known as class method)

1) Java static variable

If you declare any variable as static, it is known static variable.

- The static variable can be used to refer the common property of all objects (that is not unique for each object) e.g. company name of employees, college name of students etc.
- The static variable gets memory only once in class area at the time of class loading.

Example:

```
static String College_name="GrowMore";
```

2) Java static method

If you apply static keyword with any method, it is known as static method.

- A static method belongs to the class rather than object of a class.
- A static method can be invoked without the need for creating an instance of a class.
- static method can access static data member and can change the value of it.

Example:

1. static in getMax(int x,int y);
2. class Calculate

```
{
    static int cube(int x)
    {
        return x*x*x;
    }
    public static void main(String args[])
    {
```

```
        int result=Calculate.cube(5);  
        System.out.println(result);  
    }  
}
```

Restrictions for static method

There are two main restrictions for the static method. They are:

1. The static method can not use non static data member or call non-static method directly.
2. this and super cannot be used in static context.

Methods Overloading

If a class have multiple methods by same name but different parameters, it is known as **Method Overloading**.

Different ways to overload the method

There are two ways to overload the method in java

1. By changing number of arguments
2. By changing the data type

1)Example of Method Overloading by changing the no. of arguments

In this example, we have created two overloaded methods, first sum method performs addition of two numbers and second sum method performs addition of three numbers.

```
1. class Calculation  
2. {  
3.     void sum(int a,int b)  
4.     {  
5.         System.out.println(a+b);  
6.     }  
7.     void sum(int a,int b,int c)  
8.     {  
9.         System.out.println(a+b+c);  
10.    }  
11.  
12.    public static void main(String args[])  
13.    {  
14.        Calculation obj=new Calculation();  
15.        obj.sum(10,10,10);  
16.        obj.sum(20,20);  
17.  
18.    }  
19. }
```

2)Example of Method Overloading by changing data type of argument

In this example, we have created two overloaded methods that differs in data type.

The first sum method receives two integer arguments and second sum method receives two double arguments.

```
1. class Calculation2
2. {
3.     void sum(int a,int b)
4.     {
5.         System.out.println(a+b);
6.     }
7.     void sum(double a,double b)
8.     {
9.         System.out.println(a+b);
10.    }
11.
12.    public static void main(String args[])
13.    {
14.        Calculation2 obj=new Calculation2();
15.        obj.sum(10.5,10.5);
16.        obj.sum(20,20);
17.
18.    }
19. }
```

Nesting of Method

A Method can be called by using only its name by another method of same calss,this is known as Nesting of Method.

Example:

```
class Largest
{
    int a,b;
    Largest(int x,in y)
    {
        a=x;
        b=y;
    }
    int findLargest()
    {
        if(a>=b)
```

```
        {
            return a;
        }
        else
        {
            return b;
        }
    }
    void display()
    {
        int ans=findLargest();           //Calling another method without using dot operator
        System.out.println("Largest value is:."+ans);

    }
}
```

Constructers

Constructor in java is a *special type of method* that is used to initialize the object.

Java constructor is *invoked at the time of object creation*. It constructs the values i.e. provides data for the object that is why it is known as constructor.

Rules for creating java constructor

There are basically two rules defined for the constructor.

1. Constructor name must be same as its class name
2. Constructor must have no explicit return type

Types of java constructors

There are two types of constructors:

1. Default constructor (no-arg constructor)
2. Parameterized constructor

Java Default Constructor

A constructor that have no parameter is known as default constructor.

Syntax of default constructor:

```
1. <class_name>()  
2. {  
3.  
4. }
```

Example of default constructor

In this example, we are creating the no-arg constructor in the Bike class. It will be invoked at the time of object creation.

```
1. class Bike1  
2. {  
3.     Bike1()  
4.     {  
5.         1. System.out.println("Bike is created");  
6.     }  
7. public static void main(String args[])  
8. {  
9.     Bike1 b=new Bike1();  
10. }  
11. }
```

Java parameterized constructor

A constructor that have parameters is known as parameterized constructor.

Why use parameterized constructor?

Parameterized constructor is used to provide different values to the distinct objects.

Example of parameterized constructor

In this example, we have created the constructor of Student class that have two parameters. We can have any number of parameters in the constructor.

```
1. class Student4  
2. {  
3.     int id;  
4.     String name;  
5.  
6.     Student4(int i,String n)  
7.     {  
8.         id = i;  
9.         name = n;  
10.    }  
11.    void display(){System.out.println(id+" "+name);
```

```
12. }
13.
14. public static void main(String args[])
15. {
16.     Student4 s1 = new Student4(111,"Karan");
17.     Student4 s2 = new Student4(222,"Aryan");
18.     s1.display();
19.     s2.display();
20. }
21. }
```

Constructor Overloading in Java

Constructor overloading is a technique in Java in which a class can have any number of constructors that differ in parameter lists. The compiler differentiates these constructors by taking into account the number of parameters in the list and their type.

Example of Constructor Overloading

```
1. class Student5
2. {
3.     int id;
4.     String name;
5.     int age;
6.     Student5(int i,String n)
7.     {
8.         id = i;
9.         name = n;
10.    }
11.    Student5(int i,String n,int a)
12.    {
13.        id = i;
14.        name = n;
15.        age=a;
16.    }
17.    void display(){System.out.println(id+" "+name+" "+age);
18. }
19.
20. public static void main(String args[])
21. {
22.     Student5 s1 = new Student5(111,"Karan");
23.     Student5 s2 = new Student5(222,"Aryan",25);
24.     s1.display();
25.     s2.display();
26. }
27. }
```

Difference between constructor and method in java

There are many differences between constructors and methods. They are given below.

Java Constructor	Java Method
Constructor is used to initialize the state of an object.	Method is used to expose behaviour of an object.
Constructor must not have return type.	Method must have return type.
Constructor is invoked implicitly.	Method is invoked explicitly.
The java compiler provides a default constructor if you don't have any constructor.	Method is not provided by compiler in any case.
Constructor name must be same as the class name.	Method name may or may not be same as class name.

Inheritance

Inheritance in java is a mechanism in which one object acquires all the properties and behaviors of parent object.

The idea behind inheritance in java is that you can create new classes that are built upon existing classes. When you inherit from an existing class, you can reuse methods and fields of parent class, and you can add new methods and fields also.

Inheritance represents the **IS-A relationship**, also known as *parent-child* relationship.

Why use inheritance in java

- For Method Overriding (so runtime polymorphism can be achieved).
- For Code Reusability.

Syntax of Java Inheritance

1. **class** Subclass-name **extends** Superclass-name
2. {
3. //methods and fields
4. }

The **extends keyword** indicates that you are making a new class that derives from an existing class.

In the terminology of Java, a class that is inherited is called a super class. The new class is called a subclass.

Understanding the simple example of inheritance

1. **class** Employee
2. {
3. **float** salary=40000;

```

4. }
5. class Programmer extends Employee
6. {
7.     int bonus=10000;
8.     public static void main(String args[])
9.     {
10.        Programmer p=new Programmer();
11.        System.out.println("Programmer salary is:"+p.salary);
12.        System.out.println("Bonus of Programmer is:"+p.bonus);
13.    }
14. }

```

Types of Inheritance

On the basis of class, there can be three types of inheritance in java: single, multilevel and hierarchical.

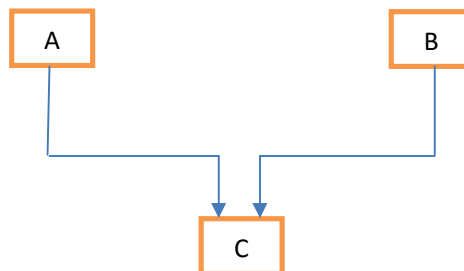
1. Single Inheritance-Only One Super Class



2. Multiple Inheritance-Several Super CLasses

When a class extends multiple classes i.e. known as multiple inheritance.

Multiple inheritance is not supported in java through class.



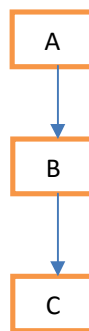
3. Hierarchichal/Hybrid Inheritance-One Superclass,Mnay Subclasses

In java programming, multiple and hybrid inheritance is supported through interface only.





4. Multilevel Inheritance- Derived from derived class



Single Inheritance Example

When a class inherits another class, it is known as a *single inheritance*.

In the example given below, Dog class inherits the Animal class, so there is the single inheritance.

File: TestInheritance.java

```
1. class Animal
2. {
3.     void eat()
4.     {
5.         System.out.println("eating...");
6.     }
7. }
8. class Dog extends Animal
9. {
10.    void bark()
```

```
11. {  
12. System.out.println("barking...")  
13. ;}  
14. }  
15. class TestInheritance  
16. {  
17. public static void main(String args[])  
18. {  
19. Dog d=new Dog();  
20. d.bark();  
21. d.eat();  
22. }  
23. }
```

Output:

```
barking...  
eating...
```

Multilevel Inheritance Example

When there is a chain of inheritance, it is known as *multilevel inheritance*. As you can see in the example given below, BabyDog class inherits the Dog class which again inherits the Animal class, so there is a multilevel inheritance.

File: TestInheritance2.java

```
1. class Animal  
2. {  
3. void eat()  
4. {  
5. System.out.println("eating...");  
6. }  
7. }  
8. class Dog extends Animal  
9. {  
10. void bark()  
11. {  
12. System.out.println("barking...");  
13. }
```

```
14. }
15. class BabyDog extends Dog
16. {
17. void weep()
18. {
19. System.out.println("weeping...");
20. }
21. }
22. class TestInheritance2
23. {
24. public static void main(String args[])
25. {
26. BabyDog d=new BabyDog();
27. d.weep();
28. d.bark();
29. d.eat();
30. }
31. }
```

Output:

```
weeping...
barking...
eating...
```

Hierarchical Inheritance Example

When two or more classes inherits a single class, it is known as *hierarchical inheritance*.

In the example given below, Dog and Cat classes inherits the Animal class, so there is hierarchical inheritance.

File: TestInheritance3.java

```
1. class Animal
2. {
3. void eat()
4. {
5. System.out.println("eating...");
6. }
7. }
```

```
5-8. class Dog extends Animal
9. {
6-10.     void bark()
7-11.     {
8-12.         System.out.println("barking...");
13. }
14. }
9-15.     class Cat extends Animal
16. {
10-17.     void meow()
11-18.     {
12-19.         System.out.println("meowing...");
20. }
21. }
13-22.     class TestInheritance3
23. {
14-24.     public static void main(String args[])
25. {
26.     Cat c=new Cat();
27.     c.meow();
28.     c.eat();
29.     //c.bark();//C.T.Error
30. }}
```

Output:

```
meowing...
eating...
```

Method Overriding

If subclass (child class) has the same method as declared in the parent class, it is known as **method overriding in java**.

In other words, If subclass provides the specific implementation of the method that has been provided by one of its parent class, it is known as method overriding.

Usage of Java Method Overriding

- Method overriding is used to provide specific implementation of a method that is already provided by its super class.
- Method overriding is used for runtime polymorphism

Rules for Java Method Overriding

1. method must have same name as in the parent class
2. method must have same parameter as in the parent class.

Example of method overriding

In this example, we have defined the run method in the subclass as defined in the parent class but it has some specific implementation. The name and parameter of the method is same and there is IS-A relationship between the classes, so there is method overriding.

```

1. class Vehicle{
2. void run(){System.out.println("Vehicle is running");}
3. }
4. class Bike2 extends Vehicle{
5. void run(){System.out.println("Bike is running safely");}
6. }
7. public static void main(String args[]){
8. Bike2 obj = new Bike2();
9. obj.run();
10. }

```

Difference between method overloading and method overriding in java

There are many differences between method overloading and method overriding in java. A list of differences between method overloading and method overriding are given below:

Method Overloading	Method Overriding
Method overloading is used <i>to increase the readability</i> of the program.	Method overriding is used <i>to provide the specific implementation</i> of the method that is already provided by its super class.
Method overloading is performed <i>within class</i> .	Method overriding occurs <i>in two classes</i> that have IS-A (inheritance) relationship.
In case of method overloading, <i>parameter must be different</i> .	In case of method overriding, <i>parameter must be same</i> .
Method overloading is the example of <i>compile time polymorphism</i> .	Method overriding is the example of <i>run time polymorphism</i> .
In java, method overloading can't be performed by changing return type of the method only. <i>Return type can be</i>	<i>Return type must be same or covariant</i> in method overriding.

same or different in method overloading. But you must have to change the parameter.

Java Method Overloading example

```
1. class OverloadingExample
2. {
3.     static int add(int a,int b)
4.     {
5.         1. return a+b;
6.     }
7.     static int add(int a,int b,int c)
8.     {
9.         return a+b+c;
10.    }
11. }
```

Java Method Overriding example

```
1. class Animal
2. {
3.     void eat()
4.     {
5.         1. System.out.println("eating...");
6.     }
7. }
8. class Dog extends Animal
9. {
10.    void eat()
11.    {
12.        System.out.println("eating bread...");
13.    }
14. }
```

final Keyword

The **final keyword** in java is used to restrict the user. The java final keyword can be used in many context. Final can be:

1. variable
2. method
3. class

1) Java final variable

If you make any variable as final, you cannot change the value of final variable(It will be constant).

Example of final variable

There is a final variable speedlimit, we are going to change the value of this variable, but It can't be changed because final variable once assigned a value can never be changed.

```
1. class Bike9
2. {
3.     final int speedlimit=90;//final variable
4.     void run()
5.     {
6.         speedlimit=400;
7.     }
8.     public static void main(String args[])
9.     {
10.        Bike9 obj=new Bike9();
11.        obj.run();
12.    }
13. }//end of class
```

2) Java final method

If you make any method as final, you cannot override it.

Example of final method

```
1. class Bike
2. {
3.     final void run()
4.     {
5.         System.out.println("running");}
6. }
7.
8. class Honda extends Bike
9. {
10.    void run()
11.    {
12.        System.out.println("running safely with 100kmph");
13.    }
14.
15.    public static void main(String args[])
16.    {
17.        Honda honda= new Honda();
18.        honda.run();
19.    }
20. }
```

3) Java final class

If you make any class as final, you cannot extend it.

Example of final class

```
1. final class Bike
2. {
3. }
4.
5. class Honda1 extends Bike
6. {
7.     void run()
8. {
9.     System.out.println("running safely with 100kmph");
10. }
11.
12. public static void main(String args[])
13. {
14.     Honda1 honda= new Honda();
15.     honda.run();
16. }
17. }
```

abstract keyword

Abstraction is a process of hiding the implementation details and showing only functionality to the user.

Another way, it shows only important things to the user and hides the internal details for example sending sms, you just type the text and send the message. You don't know the internal processing about the message delivery.

Abstraction lets you focus on what the object does instead of how it does it.

Ways to achieve Abstraction

There are two ways to achieve abstraction in java

1. Abstract class (0 to 100%)
2. Interface (100%)

Abstract class in Java

A class that is declared as abstract is known as **abstract class**. It needs to be extended and its method implemented. It cannot be instantiated.

Example abstract class

```
1. abstract class A
```

```
2. {  
3. }
```

abstract method

A method that is declared as abstract and does not have implementation is known as abstract method.

Example abstract method

```
1. abstract void printStatus();//no body and abstract
```

Example of abstract class that has abstract method

In this example, Bike the abstract class that contains only one abstract method run. Its implementation is provided by the Honda class.

```
1. abstract class Bike  
2. {  
3.   abstract void run();  
4. }  
5.  
6. class Honda4 extends Bike  
7. {  
8.   void run()  
9.   {  
10.    System.out.println("running safely..");  
11.   }  
12.  
13. public static void main(String args[])  
14. {  
15.   Bike obj = new Honda4();  
16.   obj.run();  
17. }  
18. }
```

Access Modifiers(Visibility Controls)

The access modifiers in java specify accessibility (scope) of a data member, method, constructor or class.

There are 4 types of java access modifiers:

1. private
2. default
3. protected
4. public

There are many non-access modifiers such as static, abstract, synchronized, native, volatile, transient etc. Here, we will learn access modifiers.

Access Modifier-> Access Location	Public	Protected	Friendly(Default)	Private Protected	Private
Same Class	Yes	Yes	Yes	Yes	Yes
Sub Class in Same Package	Yes	Yes	Yes	Yes	NO
Other Classes in same Package	Yes	Yes	Yes	NO	NO
Sub Class in other Packages	Yes	Yes	NO	Yes	NO
Non-Subclasses in Other Packages	Yes	NO	NO	NO	NO

1) private access modifier

The private access modifier is accessible only within class.

Simple example of private access modifier

In this example, we have created two classes A and Simple. A class contains private data member and private method. We are accessing these private members from outside the class, so there is compile time error.

```

1. class A{
2.     private int data=40;
3.     private void msg(){System.out.println("Hello java");}
4. }
5.
6. public class Simple{
7.     public static void main(String args[]){
8.         A obj=new A();
9.         System.out.println(obj.data);//Compile Time Error
10.        obj.msg();//Compile Time Error
11.    }
12. }
```

Role of Private Constructor

If you make any class constructor private, you cannot create the instance of that class from outside the class. For example:

```

1. class A{
2.     private A(){}//private constructor
3.     void msg(){System.out.println("Hello java");}
```

```

4. }
5. public class Simple{
6.     public static void main(String args[]){
7.         A obj=new A();//Compile Time Error
8.     }
9. }

```

Note: A class cannot be private or protected except nested class.

2) default access modifier

If you don't use any modifier, it is treated as **default** by default. The default modifier is accessible only within package.

Example of default access modifier

In this example, we have created two packages pack and mypack. We are accessing the A class from outside its package, since A class is not public, so it cannot be accessed from outside the package.

```

1. //save by A.java
2. package pack;
3. class A
4. {
5.     void msg()
6.     {
7.         System.out.println("Hello");
8.     }
9. }
1. //save by B.java
2. package mypack;
3. import pack.*;
4. class B
5. {
6.     public static void main(String args[])
7.     {
8.         A obj = new A();//Compile Time Error
9.         obj.msg();//Compile Time Error
10.    }
11. }

```

In the above example, the scope of class A and its method msg() is default so it cannot be accessed from outside the package.

3) protected access modifier

The **protected access modifier** is accessible within package and outside the package but through inheritance only.

The protected access modifier can be applied on the data member, method and constructor. It can't be applied on the class.

Example of protected access modifier

In this example, we have created the two packages pack and mypack. The A class of pack package is public, so can be accessed from outside the package. But msg method of this package is declared as protected, so it can be accessed from outside the class only through inheritance.

```
1. //save by A.java
2. package pack;
3. public class A{
4.     protected void msg(){System.out.println("Hello");}
5. }
1. //save by B.java
2. package mypack;
3. import pack.*;
4.
5. class B extends A{
6.     public static void main(String args[]){
7.         B obj = new B();
8.         obj.msg();
9.     }
10. }
```

Output:Hello

4) public access modifier

The **public access modifier** is accessible everywhere. It has the widest scope among all other modifiers.

Example of public access modifier

```
1. //save by A.java
2.
3. package pack;
4. public class A{
5.     public void msg(){System.out.println("Hello");}
6. }
1. //save by B.java
2.
3. package mypack;
4. import pack.*;
5.
6. class B{
7.     public static void main(String args[]){
8.         A obj = new A();
9.         obj.msg();
10.    }
11. }
```

Output:Hello